Security Implementation Quick Reference

Password Requirements

User-Facing Requirements (Show in UI)

- ✅ At least 8 characters long
- ✅ Contains uppercase letter (A-Z)
- ✅ Contains lowercase letter (a-z)
- ✅ Contains number (0-9)
- ✅ Contains special character (!@#$%^&*(),.?":{}|<>_-+=[]/;~`)
- ❌ Cannot be a common password
- ❌ Cannot contain sequential characters (123, abc)
- ❌ Cannot contain repeated characters (aaa, 111)

Password Strength Levels

- **Weak** (0-39): Red indicator, not recommended
- **Fair** (40-59): Yellow indicator, acceptable
- **Good** (60-79): Blue indicator, good choice
- **Strong** (80-100): Green indicator, excellent choice

Rate Limits

Authentication Endpoints

Endpoint	Limit	Window	Block Duration
`/api/auth/login`	5	5 min	1 hour
`/api/auth/register`	3	1 hour	-
`/api/auth/forgot-password`	3	1 hour	2 hours
`/api/auth/2fa/send-code`	5	5 min	-

API Endpoints

Endpoint	Limit	Window
`/api/listings` (GET)	30	1 min
`/api/listings` (POST)	5	5 min
`/api/upload`	10	1 min
`/api/inquiries`	5	10 min
`/api/messages`	20	5 min

Global Limits

- **Per IP**: 200 requests per minute

Error Codes

Custom Error Codes

Code	Status	Meaning
`VALIDATION_ERROR`	400	Input validation failed
`AUTHENTICATION_ERROR`	401	Login failed or token invalid
`AUTHORIZATION_ERROR`	403	Insufficient permissions
`RESOURCE_NOT_FOUND`	404	Requested resource doesn't exist
`BUSINESS_LOGIC_ERROR`	422	Business rule violation
`EXTERNAL_SERVICE_ERROR`	503	Third-party service failed
`INTERNAL_SERVER_ERROR`	500	Unexpected server error

Error Response Format

```
json
```

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Password does not meet requirements",
    "details": {
      "errors": [
        "Password must be at least 8 characters long",
        "Password must contain at least one uppercase letter"
      ]
    },
    "request_id": "a1b2c3d4-e5f6-7890-abcd-ef1234567890"
  }
}
```

Logging Levels

When to Use Each Level

- **DEBUG**: Detailed diagnostic info (dev only)
- **INFO**: General informational messages
- **WARNING**: Something unexpected but handled
- **ERROR**: Error that prevented operation
- **CRITICAL**: System failure

Example Log Entries

```
# INFO - Successful operation
2025-01-01 10:30:45 - [a1b2c3d4] - api - INFO - User logged in: user@example.com

# WARNING - Suspicious activity
2025-01-01 10:31:12 - [b2c3d4e5] - api - WARNING - Failed login attempt for: user@example.com

# ERROR - Operation failed
2025-01-01 10:32:05 - [c3d4e5f6] - api - ERROR - Failed to process payment: Stripe API error
```

Common Exceptions to Use

In Your Code

```python
# Resource not found
raise ResourceNotFoundException("Listing", listing_id)

# Invalid input
raise ValidationException("Invalid price", {"price": "must be positive"})

# Permission denied
raise AuthorizationException("You can't edit this listing")

# Authentication failed
raise AuthenticationException("Invalid credentials")

# Business logic violation
raise BusinessLogicException(
    "Free users can't create premium listings",
    {"tier": "free", "action": "create_premium"}
)

# External service failed
raise ExternalServiceException("Stripe", "Payment processing failed")
```

Frontend Integration

Display Password Strength

```javascript
const checkPassword = async (password) => {
  const response = await fetch('/api/auth/check-password-strength', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ password })
  });

  const data = await response.json();
  // data.valid, data.errors, data.score, data.strength
};
```

Handle Rate Limiting

```javascript
const handleRateLimit = (error) => {
  if (error.status === 429) {
    const retryAfter = error.headers.get('Retry-After');
    showMessage(`Too many requests. Try again in ${retryAfter} seconds.`);

    // Optional: Auto-retry after delay
    setTimeout(() => retryRequest(), retryAfter * 1000);
  }
};
```

Display Validation Errors

```javascript
const handleValidationError = (error) => {
  if (error.error.code === 'VALIDATION_ERROR') {
    const errors = error.error.details.errors;
    // Display each error near relevant form field
    errors.forEach(err => {
      showFieldError(err.field, err.message);
    });
  }
};
```

Monitoring Checklist

Daily Checks

- [ ] Review error logs for critical issues
- [ ] Check rate limit violations
- [ ] Monitor failed login attempts
- [ ] Review unusual user activity

Weekly Checks

- [ ] Analyze password strength trends
- [ ] Review rate limit effectiveness
- [ ] Check log file sizes

- [ ] Review exception patterns

Monthly Checks

- [ ] Update common password list
- [ ] Adjust rate limits based on traffic
- [ ] Review and update security policies
- [ ] Audit user permissions

Security Best Practices

DO ✅

- ✅ Always validate passwords on registration
- ✅ Log all authentication events
- ✅ Use strong, unique SECRET_KEY in production
- ✅ Apply rate limiting to all public endpoints
- ✅ Return generic error messages for auth failures
- ✅ Monitor for suspicious patterns
- ✅ Keep logs for audit trail

DON'T ❌

- ❌ Log passwords (even encrypted)
- ❌ Return detailed error messages to users
- ❌ Allow unlimited API calls
- ❌ Commit secrets to git
- ❌ Ignore rate limit violations
- ❌ Use weak or default passwords
- ❌ Disable security features in production

Testing Commands

Test Password Validation

```bash
```

```
curl -X POST http://localhost:8000/api/auth/check-password-strength \
  -H "Content-Type: application/json" \
  -d '{"password": "TestP@ssw0rd"}'
```

Test Rate Limiting

```bash
for i in {1..10}; do
  curl -X POST http://localhost:8000/api/auth/login \
    -H "Content-Type: application/json" \
    -d '{"email": "test@test.com", "password": "wrong"}' &
done
```

Check Logs

```bash
# View recent errors
tail -f logs/yachtversal_errors.log

# View all logs
tail -f logs/yachtversal.log

# Search for specific user
grep "user@example.com" logs/yachtversal.log

# Count error types
grep "ERROR" logs/yachtversal.log | wc -l
```

Emergency Procedures

High Rate of Failed Logins

1. Check logs for attacking IPs
2. Temporarily block IPs if needed
3. Review if rate limits need adjustment
4. Alert affected users if targeted attack

Password Breach Detected

1. Force password reset for affected users
2. Notify users immediately
3. Review security logs
4. Update common password blacklist

System Errors Spike

1. Check error logs for patterns
2. Review recent deployments
3. Check external service status
4. Scale resources if needed
5. Rollback if necessary

Contact Information

Development Team

- **Security Lead**: [Your Email]
- **Backend Team**: [Team Email]
- **DevOps**: [DevOps Email]

Escalation Path

1. Development Team (Response: < 1 hour)
2. Security Lead (Response: < 30 min)
3. CTO (Response: < 15 min for critical)

External Resources

- OWASP Top 10
- HaveIBeenPwned API
- FastAPI Security